

Architecture du Dépôt Git

Organisation des Branches – Phase 2

Projet Chatbot SUP'PTIC

Analyse de la situation héritée de la Phase 1,
recommandations sur les branches existantes
et nouvelle organisation pour la Phase 2

Table des matières

1	Bilan du Dépôt Héritage de la Phase 1	2
1.1	Rappel : Branches créées en Phase 1	2
1.2	Répartition des équipes Phase 1 vs Phase 2	2
2	Que Faire des Branches de la Phase 1 ?	3
2.1	Analyse branche par branche	3
2.1.1	Branches protégées : <code>main</code> et <code>dev</code>	3
2.1.2	Branches d'équipe : <code>backend</code> , <code>frontend</code> , <code>database</code> , <code>data</code>	3
2.1.3	Procédure d'archivage des branches Phase 1	3
3	Architecture des Branches Phase 2	5
3.1	Vue d'ensemble	5
3.2	Création des branches Phase 2	5
4	Arborescence complète du dépôt Phase 2	6

Bilan du Dépôt Héritage de la Phase 1

1.1 Rappel : Branches créées en Phase 1

À l'issue de la Phase 1 (8-13 février 2026), le dépôt GitHub contenait les branches suivantes :

Branch	Type	Rôle en Phase 1
main	Protégée	Version stable finale, prête pour démo
dev	Protégée	Intégration de toutes les features
backend	Équipe	API Django REST + algorithme TF-IDF
frontend	Équipe	Interface HTML/CSS/JS
database	Équipe	Modèles Django, migrations, scripts
data	Équipe	Génération et nettoyage CSV (1000+ Q/R)

Bilan Phase 1 Ce qui a été livré

- ✓ Recherche TF-IDF avec similarité cosinus opérationnelle
- ✓ Base de données Django avec 1000+ FAQ chargées
- ✓ API REST complète (5 endpoints principaux)
- ✓ Interface web responsive + système de feedback
- ✓ Application PWA avec fonctionnalités offline
- ✓ Documentation complète (backend, frontend, PWA)

1.2 Répartition des équipes Phase 1 vs Phase 2

Équipe	Phase 1	Phase 2	Évolution
Backend	2 membres	3 membres	↑ +1
Base de Données	2 membres	1 membre	↓ -1
Structuration Données	4 membres	4 membres	= Stable
Frontend	2 membres	2 membres	= Stable
Total	10	10	

Analyse de la nouvelle répartition

Le changement principal entre les deux phases est le **renforcement du Backend** (+1 membre) compensé par une **réduction de l'équipe Base de Données** (-1 membre). Cette décision est cohérente avec l'état du projet : les modèles de données étant désormais stables, les efforts doivent se concentrer sur les nouvelles fonctionnalités applicatives.

2 Que Faire des Branches de la Phase 1 ?

2.1 Analyse branche par branche

Avant de définir l'architecture de la Phase 2, il convient d'examiner chaque branche héritée et de décider de son sort.

2.1.1 Branches protégées : main et dev

■ À conserver impérativement

- **main** Contient la version stable 1.0.0. À garder comme référence historique et point de départ de la Phase 2.
 - **dev** Sert de branche d'intégration continue. Elle est réutilisée telle quelle en Phase 2.
- Ces deux branches ne doivent **jamais** être supprimées.

2.1.2 Branches d'équipe : backend, frontend, database, data

■ Décision recommandée pour les branches d'équipe

Proposition : Archiver, puis recréer proprement.

Les branches d'équipe de la Phase 1 portent l'historique de commits de tout le sprint précédent. Continuer à travailler directement dessus en Phase 2 risque de :

- ✗ Créer de la confusion entre les travaux des deux phases
- ✗ Polluer l'historique Git avec des commits mélangés
- ✗ Rendre difficile la revue de code phase par phase

Procédure recommandée :

1. Créer un **tag d'archivage** sur chaque branche Phase 1
2. Supprimer les anciennes branches sur GitHub
3. Recréer des branches fraîches pour la Phase 2 (à partir de **dev**)

2.1.3 Procédure d'archivage des branches Phase 1

Listing 1 – Archivage des branches Phase 1 avant suppression

```
# Etape 1 : Se positionner sur chaque branche et creer un tag
git checkout backend
git tag archive/phase1/backend
git push origin archive/phase1/backend

git checkout frontend
git tag archive/phase1/frontend
git push origin archive/phase1/frontend

git checkout database
git tag archive/phase1/database
git push origin archive/phase1/database

git checkout data
```

```
git tag archive/phase1/data
git push origin archive/phase1/data

# Etape 2 : Supprimer les branches distantes
git push origin --delete backend
git push origin --delete frontend
git push origin --delete database
git push origin --delete data

# Etape 3 : Nettoyer les branches locales
git branch -D backend
git branch -D frontend
git branch -D database
git branch -D data
```

3 Architecture des Branches Phase 2

3.1 Vue d'ensemble

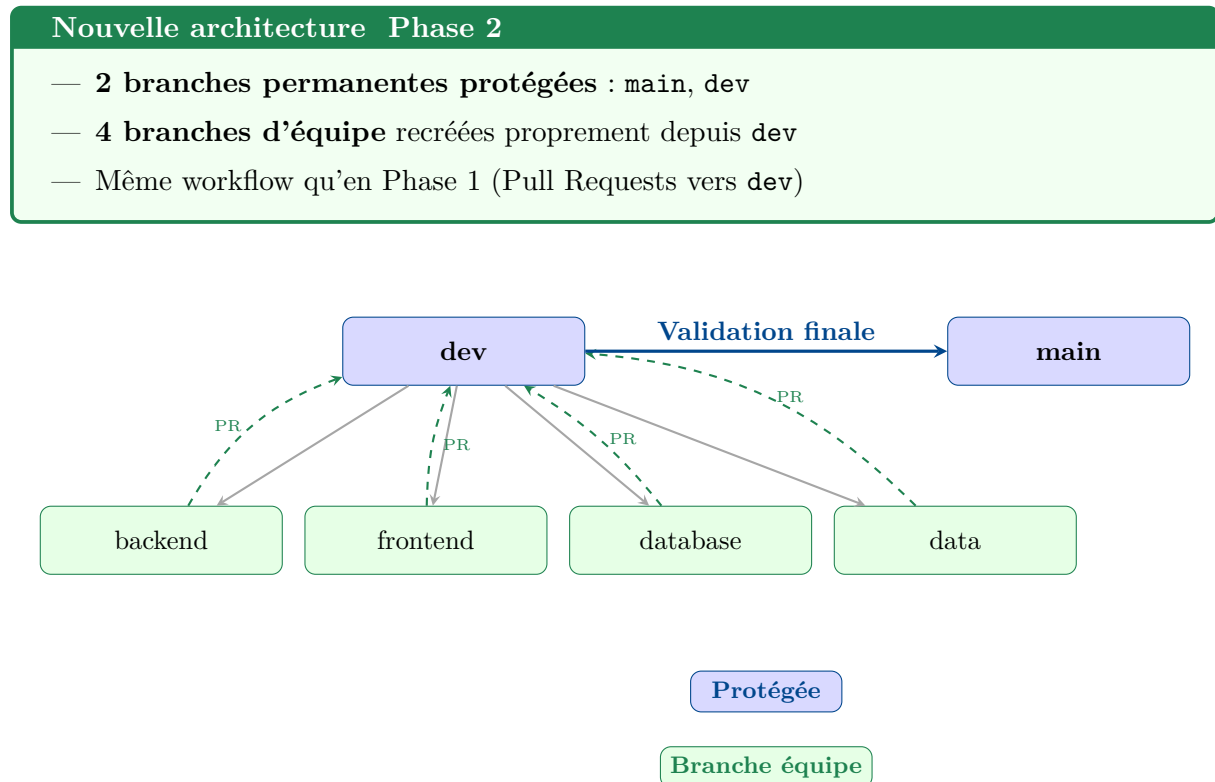


FIGURE 1 – Flux de travail Git Phase 2

3.2 Création des branches Phase 2

Listing 2 – Initialisation des branches Phase 2 depuis dev

```
# S'assurer d'être à jour
git checkout dev
git pull origin dev

# Creer les nouvelles branches Phase 2 depuis dev
git checkout -b backend
git push origin backend

git checkout dev
git checkout -b frontend
git push origin frontend

git checkout dev
git checkout -b database
git push origin database

git checkout dev
git checkout -b data
git push origin data
```

4 Arborescence complète du dépôt Phase 2

Légende

Les éléments marqués # [NEW] sont nouveaux par rapport à la Phase 1.
Les dossiers backend/rag_data/ contiennent des artefacts générés localement et ne sont pas commités (dans .gitignore).

Listing 3 – Arborescence Phase 2

```
ChatBot/                                     # Racine
|
+-- .venv/                                   # Env virtuel Python
+-- .gitignore / requirements.txt           # Dépendances (Phase 2 ajoutées)
+-- README.md
|
+-- backend/                                # Django
|   +-- config/                             # Paramètres + routes [NEW]
|   +-- faq/                               # FAQ (hérité)
|   +-- chatbot/                             # Moteur RAG
|       |   +-- views.py                     # /api/chatbot/ask/ [NEW]
|       |   +-- engine/ [NEW]                # Pipeline RAG
|       |       +-- embedder.py              # MiniLM
|       |       +-- faiss_search.py          # FAISS
|       |       +-- llm_client.py            # Ollama Phi-3
|       |       +-- rag_pipeline.py          # Orchestration
|       |       +-- tfidf_fallback.py        # Bascules TF-IDF
|       |       \-- prompt_builder.py        # Prompt LLM
|   +-- users/                              # Utilisateurs + feedback
|   +-- rag_data/ [NEW]                     # Artefacts FAISS
|   +-- scripts/ [NEW]                     # build_index, reload_index, test_rag
|   +-- doc/                                # Docs backend
|   +-- manage.py / db.sqlite3
|
+-- data/                                   # Données FAQ JSON
|   +-- faq_*.json                          # Admissions, frais, etc.
|   +-- scripts/                            # validate_json, stats_dataset [NEW]
|   \-- doc/
|
+-- frontend/                              # Interface
|   +-- react-app/ [NEW]                    # App React
|   +-- pwa/                                # Ancienne PWA (archivée)
|   +-- README.md / start_server.py
|
+-- docs/                                   # Documentation générale
|   +-- architecture/
|   \-- setup/                             # OLLAMA_SETUP.md [NEW]
```

Document préparé par :
NKOUMOU TJADE
 Chef de la Division de Programmation
 Club Informatique SUP'PTIC